

EFFICIENT ROM SIZE REDUCTION FOR DISTRIBUTED ARITHMETIC

Jung-Pil Choi

Seung-Cheol Shin

Jin-Gyun Chung

Dept. of Information & Communication Engr., Chonbuk National University, Chonju, Korea
{jgchung}@moak.chonbuk.ac.kr

ABSTRACT

In distributed arithmetic-based architecture for an inner product between two length- N vectors, the size of the ROM increases exponentially with N . Moreover, the ROMs are generally the bottleneck of speed, especially when their size is large. In this paper, a ROM size reduction technique for DA (Distributed Arithmetic) is proposed. The proposed method is based on modified OBC (Offset Binary Coding) and control circuit reduction technique. By simulations, it is shown that the use of the proposed technique can result in reduction in the number of gates up to 50%.

1 INTRODUCTION

In DA (Distributed Arithmetic)-based architecture for an inner product between two vectors, the multiplications are reordered and mixed such that the arithmetic becomes distributed through the structure [1]. DA is commonly used for implementation of convolution operations and discrete cosine transforms [2]-[3]. However, inner product operation often requires very large ROM area and processing time especially when the vector length N is large.

In [4], OBC (Offset Binary Coding) was proposed to reduce the ROM size. In OBC scheme, the content of the ROM is rearranged such that the whole ROM table can be represented by only lower (or, upper) half of the new ROM table. Thus the ROM table can be reduced by a factor of 2 with some control circuit overhead.

In this paper, new ROM size reduction method is proposed. By repeatedly applying the proposed method, the size of the ROM table can be reduced by a factor of 2 to 2^{N-1} . In addition, a control circuit reduction technique is proposed.

In section 2, DA and DA-OBC are briefly reviewed. New ROM size reduction method is proposed in sections 3 and 4. In section 5, OBC scheme and the proposed scheme are compared in terms of the number of transistors needed to implement an inner-product operation.

2 DA AND DA-OBC

2.1 DA

Consider an inner product between two length- N vectors C and X :

$$Y = \sum_{i=0}^{N-1} c_i x_i. \quad (1)$$

In (1), $\{c_i\}$'s are M -bit constants and $\{x_i\}$'s are input data coded as W -bit 2's complement numbers. Then, (1) can be expressed as

$$Y = \sum_{i=0}^{N-1} c_i \left(-x_{i,W-1} + \sum_{j=1}^{W-1} x_{i,W-1-j} 2^{-j} \right), \quad (2)$$

$$= - \sum_{i=0}^{N-1} c_i x_{i,W-1} + \sum_{j=1}^{W-1} \left(\sum_{i=0}^{N-1} c_i x_{i,W-1-j} \right) 2^{-j}. \quad (3)$$

By defining

$$C_{W-1-j} = \sum_{i=0}^{N-1} c_i x_{i,W-1-j} \quad (j \neq 0), \quad (4)$$

$$C_{W-1} = - \sum_{i=0}^{N-1} c_i x_{i,W-1}, \quad (5)$$

(3) becomes

$$Y = \sum_{j=0}^{W-1} C_{W-1-j} 2^{-j}. \quad (6)$$

In (4) and (5), since C_{W-1-j} depends on $x_{i,W-1-j}$ and has only 2^N possible values, it is possible to precompute them and store them in a ROM. Table 1 shows the content of the ROM for $N=4$. A typical architecture for the computation of an inner product of two length- N vectors using DA is shown in Fig. 1. In Fig. 1, an input set of N bits $\{x_{i,W-1-j}, i = 0, 1, \dots, N-1\}$ is used as an address to read the corresponding C_{W-1-j} value from the ROM. The result Y is obtained by accumulating shifted C_{W-1-j} values according to (6). In Fig. 1, the computation runs from $j=0$ to $j=W-1$ and the control signal S is 1 when $j=W-1$, and 0 otherwise.

2.2 DA-OBC

In OBC scheme, x_i is rewritten as;

$$\begin{aligned} x_i &= \frac{1}{2} [x_i - (-x_i)], \\ &= \frac{1}{2} [-(x_{i,W-1} - \overline{x_{i,W-1}}) + \sum_{j=1}^{W-1} (x_{i,W-1-j} - \overline{x_{i,W-1-j}}) 2^{-j} - 2^{-(W-1)}]. \end{aligned} \quad (7)$$

Define

Table 1. Content of the ROM of DA

$x_{0,j}$	$x_{1,j}$	$x_{2,j}$	$x_{3,j}$	Content of the ROM
0	0	0	0	0
0	0	0	1	c_3
0	0	1	0	c_2
0	0	1	1	$c_2 + c_3$
0	1	0	0	c_1
0	1	0	1	$c_1 + c_3$
0	1	1	0	$c_1 + c_2$
0	1	1	1	$c_1 + c_2 + c_3$
1	0	0	0	c_0
1	0	0	1	$c_0 + c_3$
1	0	1	0	$c_0 + c_2$
1	0	1	1	$c_0 + c_2 + c_3$
1	1	0	0	$c_0 + c_1$
1	1	0	1	$c_0 + c_1 + c_3$
1	1	1	0	$c_0 + c_1 + c_2$
1	1	1	1	$c_0 + c_1 + c_2 + c_3$

Table 2. Content of the ROM of DA-OBC

$x_{0,j}$	$x_{1,j}$	$x_{2,j}$	$x_{3,j}$	Content of the ROM
0	0	0	0	$-(c_0 + c_1 + c_2 + c_3)/2$
0	0	0	1	$-(c_0 + c_1 + c_2 - c_3)/2$
0	0	1	0	$-(c_0 + c_1 - c_2 + c_3)/2$
0	0	1	1	$-(c_0 + c_1 - c_2 - c_3)/2$
0	1	0	0	$-(c_0 - c_1 + c_2 + c_3)/2$
0	1	0	1	$-(c_0 - c_1 + c_2 - c_3)/2$
0	1	1	0	$-(c_0 - c_1 - c_2 + c_3)/2$
0	1	1	1	$-(c_0 - c_1 - c_2 - c_3)/2$
1	0	0	0	$(c_0 - c_1 - c_2 - c_3)/2$
1	0	0	1	$(c_0 - c_1 - c_2 + c_3)/2$
1	0	1	0	$(c_0 - c_1 + c_2 - c_3)/2$
1	0	1	1	$(c_0 - c_1 + c_2 + c_3)/2$
1	1	0	0	$(c_0 + c_1 - c_2 - c_3)/2$
1	1	0	1	$(c_0 + c_1 - c_2 + c_3)/2$
1	1	1	0	$(c_0 + c_1 + c_2 - c_3)/2$
1	1	1	1	$(c_0 + c_1 + c_2 + c_3)/2$

$$d_{i,j} = \begin{cases} x_{i,j} - \overline{x_{i,j}}, & \text{for } j \neq W-1 \\ -(x_{i,W-1} - \overline{x_{i,W-1}}), & \text{for } j = W-1 \end{cases} \quad (8)$$

and $d_{i,j} \in \{-1, +1\}$. Using (7) and (8), (1) can be rewritten as

$$\begin{aligned} Y &= \sum_{i=0}^{N-1} \frac{1}{2} c_i \left[\sum_{j=0}^{W-1} d_{i,W-1-j} 2^{-j} - 2^{-(W-1)} \right], \\ &= \sum_{j=0}^{W-1} \left(\sum_{i=0}^{N-1} \frac{1}{2} c_i d_{i,W-1-j} \right) 2^{-j} - \left(\frac{1}{2} \sum_{i=0}^{N-1} c_i \right) 2^{-(W-1)} \end{aligned}$$

By defining

$$\begin{aligned} D_j &= \sum_{i=0}^{N-1} \frac{1}{2} c_i d_{i,W-1-j}, \text{ for } 0 \leq j \leq W-1, \\ D_{extra} &= -\frac{1}{2} \sum_{i=0}^{N-1} c_i, \end{aligned} \quad (10)$$

(9) can be rewritten as

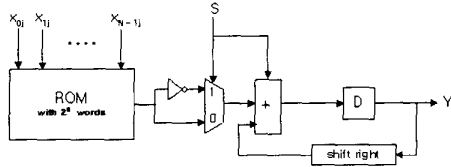


Figure 1. Architecture of computing inner product of two length- N vectors using distributed arithmetic.

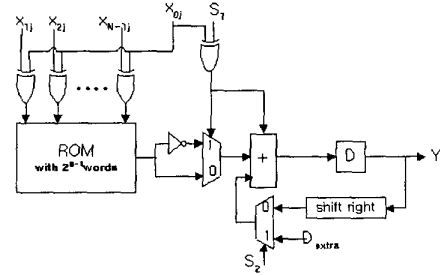


Figure 2. Architecture of computing inner product of two length- N vectors using DA with OBC coding.

$$Y = \sum_{j=0}^{W-1} D_{W-1-j} 2^{-j} + D_{extra} 2^{-(W-1)}. \quad (11)$$

The OBC scheme is characterized by (10) and (11). Table 2 shows the content of the ROM for $N=4$. In Table 2, notice that the upper-half and the lower-half ROM values are mirrored. Therefore, it is possible to reduce the ROM size by a factor of 2. Fig. 2 shows a typical architecture for the computation of an inner product of two length- N vectors using OBC scheme. In Fig. 2, two control signals S_1 and S_2 are required, where S_1 is 1 when $j = W-1$ and 0 otherwise, and S_2 is 1 when $j = 0$ and 0 otherwise.

3 MODIFIED OBC ARCHITECTURE

As was discussed in section 2.2, only upper-half of the original ROM table is needed in OBC scheme. Table 3 shows the content of the ROM with OBC scheme ($N=4$). It can be observed from Table 3 that the ROM values except c_0 term are mirrored along the line between the 4-th and the 5-th rows. In other words, except c_0 term, the ROM values in Table 3 have only 2^{N-2} possible values depending on the input $x_{i,j}$ values. Therefore, the ROM table of OBC scheme can be reduced by a factor of 2. Table 4 illustrates the new ROM table and Fig. 3 shows the architecture for the computation of inner product using the proposed method. By

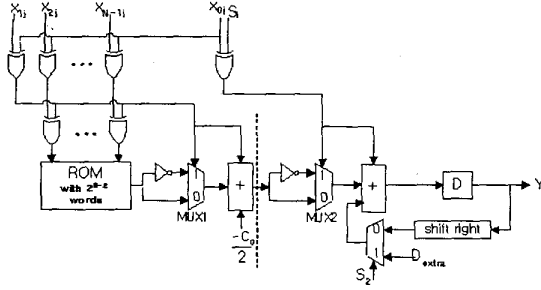


Figure 3. Reduction of ROM part of Fig. 2.

Table 3. Content of the ROM in Fig 2

$x_{1,j}$	$x_{2,j}$	$x_{3,j}$	Content of the ROM
0	0	0	$-c_0/2 - (c_1 + c_2 + c_3)/2$
0	0	1	$-c_0/2 - (c_1 + c_2 - c_3)/2$
0	1	0	$-c_0/2 - (c_1 - c_2 + c_3)/2$
0	1	1	$-c_0/2 - (c_1 - c_2 - c_3)/2$
1	0	0	$-c_0/2 + (c_1 - c_2 - c_3)/2$
1	0	1	$-c_0/2 + (c_1 - c_2 + c_3)/2$
1	1	0	$-c_0/2 + (c_1 + c_2 - c_3)/2$
1	1	1	$-c_0/2 + (c_1 + c_2 + c_3)/2$

repeatedly applying the proposed method, the ROM size can be reduced upto 2 words.

By the proposed method, ROM size reduction is achieved with the overhead of control circuits such as XOR (exclusive OR), MUX (multiplexer), INV (inverter), and FA (full adder) as shown in Fig. 3. While the increase in the number of XOR gates is proportional to the input vector length N , the complexities of other control circuits (MUX, INV and FA) increase in proportion to the coefficient word-length. To achieve the size reduction of the whole system, the reduction in the number of ROM cells and the decoder circuit inside the ROM should be larger than the hardware increase by the control circuits. A control circuit reduction method is described in section 4.

In addition, due to the additional MUX, XOR, INV and FA circuits, the critical path of the proposed architecture is increased. If desired, the critical path can be reduced by placing pipeline registers at appropriate feed-forward cut-set locations [5].

4 CIRCUIT COMPLEXITY ANALYSIS OF THE PROPOSED ARCHITECTURE

In this section, hardware complexity of each component in the proposed architecture is analyzed in terms of the number of transistors. Also, a control circuit minimization technique is proposed.

In this section, N_v , N_i and N_c represents the input vector length, the input vector word-length, and the coefficient word-length, respectively. Also, it is assumed that AND,

Table 4. Content of the Reduced ROM

$x_{1,j}$	$x_{2,j}$	$x_{3,j}$	Content of the ROM
0	0	0	$-(c_1 + c_2 + c_3)/2$
0	0	1	$-(c_1 + c_2 - c_3)/2$
0	1	0	$-(c_1 - c_2 + c_3)/2$
0	1	1	$-(c_1 - c_2 - c_3)/2$

XOR, MUX and INV is constructed using 4, 8, 6 and 2 transistors, respectively.

4.1 Decoder

The size of the decoder inside a ROM depends on N_v and is independent of N_i and N_c . The number of AND gates for the decoder of OBC scheme is

$$D_{OBC} = 2 \times \sum_{i=2}^{N_v-1} (N_v - i) + 2^{N_v-1}. \quad (12)$$

On the other hand, the number of AND gates for the decoder of the proposed scheme is

$$D_{mod} = 2 \times \sum_{i=3}^{N_v-1} (N_v - i) + 2^{N_v-2}. \quad (13)$$

From (12) and (13),

$$\begin{aligned} D_{OBC} - D_{mod} &= 2 \sum_{i=2}^{N_v-1} (N_v - i) + 2^{N_v-1} - [2 \sum_{i=3}^{N_v-1} (N_v - i) + 2^{N_v-2}] \\ &= 2(N_v - 2) + 2^{N_v-1}. \end{aligned} \quad (14)$$

Therefore, the number of the transistors reduced by the proposed scheme is

$$(D_{OBC} - D_{mod}) \times 4(TR's) = [2(N_v + 2^{N_v-1}) \times 4(TR's)]. \quad (15)$$

4.2 ROM

The ROM size for OBC scheme is

$$R_{OBC} = 2^{N_v-1} \times N_c(TR's). \quad (16)$$

The ROM size for the proposed scheme is

$$R_{mod} = 2^{N_v-2} \times N_c(TR's). \quad (17)$$

Therefore, the number of transistors reduced by the proposed scheme is

$$(R_{OBC} - R_{mod}) \times 1(TR's) = 2^{N_v-2} \times N_c(TR's). \quad (18)$$

4.3 MUX and INV

The hardware complexity of MUX/INV depends only on the coefficient word-length and is given by

$$M_i = 6 \times N_c(TR's), \quad I_i = 2 \times N_c(TR's). \quad (19)$$

One MUX and one INV circuits are added each time the proposed algorithm is applied.

4.4 FA

The implementation of a FA requires 30TR's. Thus the number of TR's used in OBC scheme is $30 \times N_c(TR's)$. As can be seen from Fig. 3, additional N_c FA's are needed each time the proposed algorithm is applied. Since a FA requires 30TR's, its efficient reduction is very important in practical concern.

The number of TR's of the additional FA's in Fig. 3 can be reduced significantly by observing the fact that one of the three inputs of the additional FA's is always fixed. For

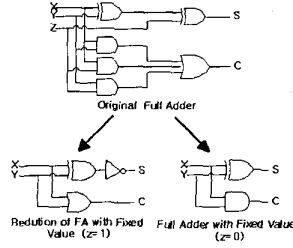


Figure 4. FA with fixed values.

example, in Fig. 3, one of the three inputs of the additional FA is fixed as $-c_0/2$. Fig. 4 shows the hardware reduction of a FA with a fixed input. For the fixed input value of 0 and 1, only 12TR's and 14TR's are needed, respectively.

Assuming that a coefficient has $N_c/2$ nonzero bits on the average, the increase in the number of TR's by the proposed scheme is

$$F_i = \frac{N_c}{2} \times 14 + \frac{N_c}{2} \times 12 = 13N_c(TR's). \quad (20)$$

4.5 XOR

Fig. 3 shows that additional $N_v - 2$ XOR gates are needed by the proposed scheme. The function of the control circuit in Fig. 3 can be summarized as shown in Table 5. Using Table 5, the control circuit in Fig. 3 can be simplified as shown in Fig. 5. Therefore, no additional XOR gates are needed by the proposed scheme.

5 COMPARISONS BETWEEN OBC AND MODIFIED OBC

When OBC is modified by the proposed scheme, the total number of TR's is increased by $M_i + I_i + F_i - (D_r + R_r)$. Therefore, to achieve hardware reduction by the proposed scheme, the following condition should be satisfied:

$$D_r + R_r > M_i + I_i + F_i. \quad (21)$$

Fig. 6 shows the size reduction rate of the proposed scheme (Fig. 5) over OBC (Fig. 2) for various values of N_c and N_v . Hardware reduction by the proposed scheme is obtained when N_v is larger than 5. As N_v increases, the reduction rate approaches 50% for all values of N_c . Fig. 6 shows the size reduction rate for $N_c = 8$, and $N_v = 4 \sim 16$. Table 6 compares OBC and modified OBC in terms of the number of TR's for $N_c = 8$, and $N_v = 8$.

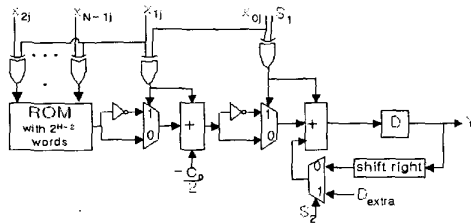


Figure 5. Reduction of control block in Fig. 3.

Table 5. Relation between input and output of control block

ROM input	MUX1 control input	MUX2 control input
$x_{0,j} = 0, x_{1,j} = 0$	$x_{2,j}, x_{3,j}$	non inverting
$x_{0,j} = 0, x_{1,j} = 1$	$x_{2,j}, x_{3,j}$	inverting
$x_{0,j} = 1, x_{1,j} = 0$	$x_{2,j}, x_{3,j}$	inverting
$x_{0,j} = 1, x_{1,j} = 1$	$x_{2,j}, x_{3,j}$	non inverting

Table 6. Size comparison for $N_v = N_c = 8$

Logic	Architecture using OBC (Fig 2)	Modified Architecture (Fig 5)	Difference
Decoder	$170 \times 4 = 680$	$94 \times 4 = 396$	-284
ROM	$128 \times 8 = 1024$	$64 \times 8 = 512$	-512
XOR	$8 \times 8 = 64$	64	0
MUX	$8 \times 2 \times 6 = 96$	$96 + 8 \times 6 = 144$	+48
INV	$8 \times 2 = 16$	$16 + 8 \times 2 = 32$	+16
Register	$8 \times 16 = 128$	128	0
Shift Accumulator	$8 \times 16 = 128$	128	0
FA	$8 \times 30 = 240$	$240 + (14 \times 4 + 12 \times 4) = 344$	+104
Total	2376 TR's	1748 TR's	-628 TR's

6 CONCLUSIONS

In this paper, a ROM size reduction technique for DA was proposed based on modified OBC and control circuit reduction technique. By repeatedly applying the proposed method, the size of the ROM table can be reduced by a factor of 2 to 2^{N_v-1} . By simulations, it was shown that the use of the proposed technique can result in reduction in the number of transistors up to 50%.

REFERENCES

- [1] C. S. Burrus, "Digital filter structures described by distributed arithmetic," *IEEE Trans. on Circuits and Systems*, Dec. 1977.
- [2] M. T. Sun, T. C. Chen, and A. M. Gotlieb, "VLSI implementation of a 16×16 discrete cosine transform," *IEEE Trans. on Circuits and Systems*, vol. CAS-36, pp. 610-617, Apr. 1989.
- [3] M. T. Sun, L. Wu, and M. L. Liou, "A concurrent architecture for VLSI implementation of discrete cosine transform," *IEEE Trans. on Circuits and Systems*, vol. CAS-34, pp. 992-994, Aug. 1987.
- [4] N. Demassieux and F. Jutand, *VLSI Implementations for Image Communications* (P. Pirsch, ed.), ch. Orthogonal transforms, pp. 217-250. Elsevier, 1993.
- [5] K. K. Parhi, *VLSI Digital Signal Processing Systems*. John Wiley & Sons, Inc., 1999.

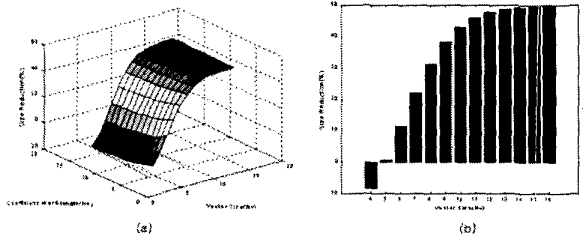


Figure 6. (a) Size reduction rate of modified OBC over OBC for various N_c and N_v values. (b) Size reduction rate for $N_c = 8$, $N_v = 4 \sim 16$.